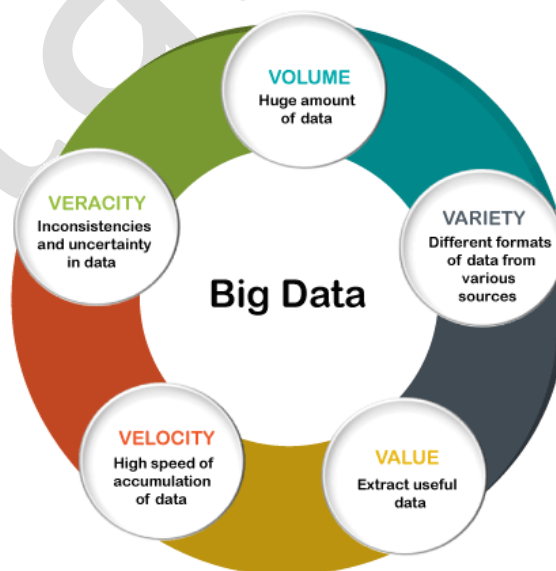


Apache Spark

- Apache Spark is a powerful open-source distributed computing system designed for big data processing and analytics. It was developed to be fast and general-purpose, with APIs for Scala, Java, Python, and R.
 - Spark use RAM instead of ROM
 - It can connect with Hadoop Distributed File System (HDFS), Java Database Connectivity (JDBC)
 - Key Features of Apache Spark:
 - Speed: Spark processes data in memory, making it much faster than traditional disk-based processing frameworks like Hadoop MapReduce. It can be up to 100 times faster for certain applications.
 - Ease of Use: Spark provides high-level APIs in Java, Scala, Python, and R, and it includes a rich set of built-in libraries. It also has interactive shells for Scala (spark-shell) and Python (pyspark).
 - General-purpose: Spark supports a wide range of workloads, including batch processing, interactive queries (Spark SQL), real-time streaming (Spark Streaming), machine learning (MLlib), and graph processing (GraphX).
 - Scalability: Spark can handle petabytes of data on large clusters of machines, making it suitable for big data applications.
 - Fault Tolerance: Spark ensures fault tolerance through its lineage data structure, which tracks transformations applied to data.
-

Big Data

- Big Data refers to the massive volumes of structured and unstructured data that are too large to process using traditional data processing techniques. This data comes from various sources such as social media, sensors, digital images, videos, transaction records, and more. The concept of Big Data encompasses not only the data itself but also the tools and technologies used to process, analyze, and extract meaningful insights from this data.
- Characteristics of Big Data (The 5 V's):
 - Volume: The sheer amount of data generated every second is enormous. Big Data technologies are designed to handle petabytes and exabytes of data.
 - Velocity: Data is being generated at an unprecedented speed and must be processed quickly. This includes real-time or near-real-time processing of streaming data.
 - Variety: Data comes in various formats – structured, semi-structured, and unstructured. Examples include text, images, videos, social media posts, and more.
 - Veracity: The quality and accuracy of the data can vary greatly, and it's crucial to validate and cleanse the data for reliable analysis.
 - Value: The most important aspect of Big Data is the value it can provide. Through proper analysis, valuable insights and actionable intelligence can be extracted from Big Data.



Monolithic Architecture vs Distributed Architecture

Aspect	Monolithic Architecture	Distributed Architecture
Definition	Single, unified application	Multiple independent services or components
Deployment	Deployed as a single unit	Deployed as separate services or components
Scalability	Vertical scaling (adding more resources to a single node)	Horizontal scaling (adding more nodes or instances)
Development	Easier to develop and manage initially	More complex to develop and manage initially
Maintenance	Can become difficult to maintain as the application grows	Easier to maintain and update individual components
Fault Tolerance	Failure in one part can affect the whole system	Failure in one component does not necessarily affect others
Performance	Generally faster for inter-component communication	Network latency can affect performance
Flexibility	Less flexible, tightly coupled components	More flexible, loosely coupled components
Technology Stack	Typically uses a single technology stack	Can use multiple technology stacks (polyglot architecture)
Deployment Speed	Slower to deploy changes (all or nothing)	Faster to deploy changes (deploy individual components)
Testing	Easier to test as a whole application	More complex testing, need to test interactions between components
Examples	Traditional enterprise applications	Microservices, cloud-native applications
Resource Utilization	May underutilize resources	Better resource utilization through distributed computing
Data Management	Centralized data management	Decentralized data management, potential data consistency issues
Communication	In-process method calls (faster)	Network calls (slower, but more resilient)

Apache Hadoop vs Apache Spark

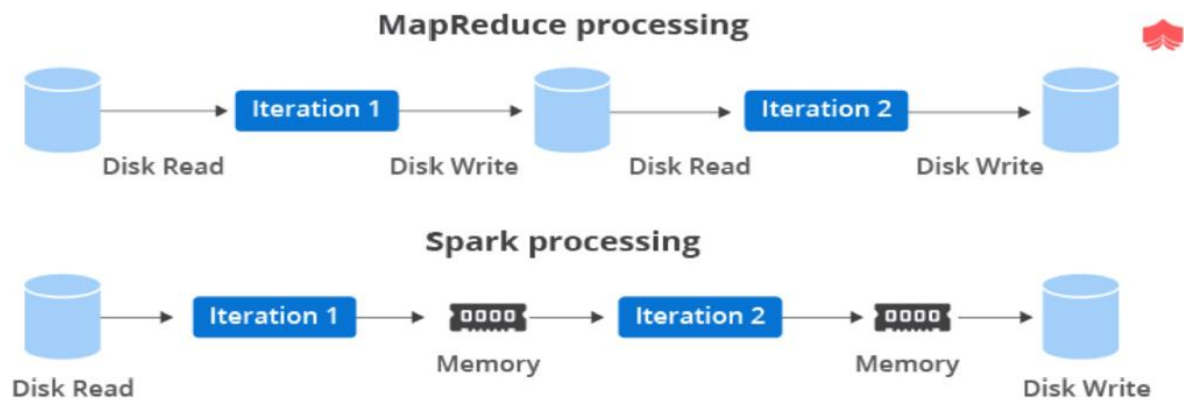
Aspect	Apache Hadoop	Apache Spark
Core Component	Hadoop Distributed File System (HDFS) and MapReduce	Spark Core
Data Processing	Disk-based processing	Memory-based processing (faster iterative algorithms)
Processing Model	Batch processing	In-memory processing (real-time and batch)
Processing Speed & Performance	Hadoop's MapReduce model reads and writes from a disk, slowing down processing speed	Spark reduces read/write cycles to disk and stores intermediate data in memory, resulting in faster processing
Ease of Use	More complex, requires Java/MapReduce programming	Easier, with high-level APIs in Scala, Java, Python, and R
Usage	Designed to handle batch processing efficiently	Designed to handle real-time data efficiently
Latency	High latency computing framework, does not have an interactive mode	Low latency computing, can process data interactively
Fault Tolerance	High, achieved by replicating blocks of data. If a node goes down, data can be found on another node	High, achieved by storing a chain of transformations. If data is lost, it can be recomputed on original data
Scalability	Highly scalable by adding nodes and disk for storage, supports tens of thousands of nodes	Quite difficult to scale as it relies on RAM for computations, supports thousands of nodes in a cluster
Real-time Processing	Not designed for real-time processing	Designed for real-time processing (Spark Streaming)
Cost	Cheaper option available in terms of cost	Requires a lot of RAM for in-memory operations, increasing the cluster and hence cost
Algorithm Used	Uses the PageRank algorithm	Uses a graph computation library called GraphX
Security	Supports LDAP, ACLs, SLAs, etc., making it extremely secure	Not secure on its own, relies on integration with Hadoop for necessary security levels
Machine Learning	Data fragments can be too large and create bottlenecks, making it slower than Spark	Much faster as it uses MLlib for computations and has in-memory processing

Language Support	Uses Java or Python for MapReduce apps	Uses Java, R, Scala, Python, or Spark SQL for the APIs
User-friendliness	More difficult to use	More user-friendly
Resource Management	YARN (Yet Another Resource Negotiator) is the most common option for resource management	Built-in tools for resource management, can use YARN, Mesos, or Kubernetes
Graph Processing	Apache Giraph	GraphX
Ecosystem Tools	Hive, Pig, HBase, etc.	Spark SQL, GraphX, MLlib, Spark Streaming
Community and Support	Large, well-established	Rapidly growing, strong support from the open-source community and enterprises
Use Cases	Batch processing, ETL jobs, data warehousing	Real-time data processing, machine learning, interactive queries
Data Sources	Primarily HDFS, but also supports S3, etc.	Supports HDFS, S3, Cassandra, HBase, etc.

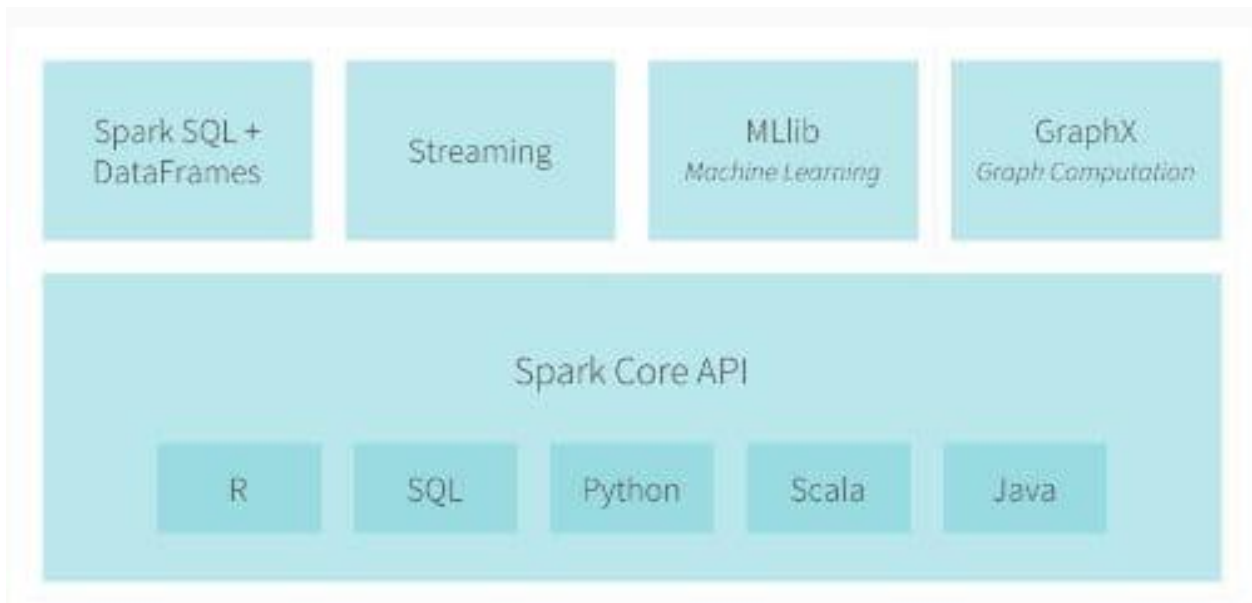
Why Apache Spark Was Developed

- Performance Improvements:
 - Disk-Based Storage Limitation: Traditional MapReduce relies on disk storage for intermediate data, which slows down processing due to frequent read/write operations.
 - In-Memory Processing: Spark introduced in-memory computation, significantly speeding up data processing tasks by reducing disk I/O.
- Ease of Use:
 - Complex APIs in MapReduce: Traditional MapReduce APIs are low-level and can be cumbersome for complex data tasks.
 - High-Level Abstractions: Spark provides higher-level APIs such as DataFrames and Spark SQL, simplifying data processing and analytics.
- Support for Advanced Analytics:
 - Iterative Algorithms: Many machine learning and graph processing tasks require iterative computations. Spark's in-memory processing is well-suited for these tasks.
 - Real-Time Processing: Spark Streaming supports real-time data processing, which is not natively supported by traditional MapReduce.
- Fault Tolerance:

- Resilient Distributed Datasets (RDDs): Spark's RDDs provide fault tolerance by allowing recovery of lost data through lineage information, making fault tolerance more efficient and flexible.
5. Flexibility and Scalability:
- Multiple Data Sources: Spark integrates with various data sources (e.g., HDFS, Hive, HBase), making it versatile for big data processing.
 - Scalability: Designed to scale out across clusters of machines, Spark handles large datasets effectively.
6. Community and Ecosystem:
- Rich Ecosystem: Spark includes libraries and components for SQL (Spark SQL), machine learning (MLlib), graph processing (GraphX), and streaming (Spark Streaming).
 - Active Development: Spark benefits from an active community and continuous improvements, staying current with advancements in big data processing.



Spark Ecosystem



- Apache Spark has a rich ecosystem that extends its core capabilities and allows it to handle a wide variety of data processing and analytics tasks. Here's a detailed overview of the components within the Spark ecosystem:

1. Spark Core

- Spark Core is the foundation of the Spark ecosystem. It provides:
 - i. Basic functionalities such as task scheduling, memory management, fault recovery, and storage system interactions.
 - ii. APIs in Java, Scala, Python, and R for developing data processing applications.
 - iii. The Resilient Distributed Dataset (RDD) abstraction, which is a fault-tolerant collection of elements that can be operated on in parallel.

2. Spark SQL

- Spark SQL is a module for working with structured data. It provides:
 - i. Support for SQL queries, DataFrames, and Datasets.
 - ii. Integration with a variety of data sources like Hive, Avro, Parquet, ORC, JSON, and JDBC.
 - iii. Catalyst optimizer for query optimization.

3. Spark Streaming

- Spark Streaming enables scalable and fault-tolerant stream processing of live data streams. Key features include:
 - i. Real-time data processing.
 - ii. Integration with various data sources like Kafka, Flume, Kinesis, and TCP sockets.
 - iii. Use of the same code for batch and streaming analytics.

4. MLlib (Machine Learning Library)

- MLlib is Spark's scalable machine learning library. It provides:
 - i. A variety of machine learning algorithms including classification, regression, clustering, collaborative filtering, and more.
 - ii. Tools for feature extraction, transformation, and selection.
 - iii. Pipelines for building, tuning, and deploying machine learning models.

5. GraphX

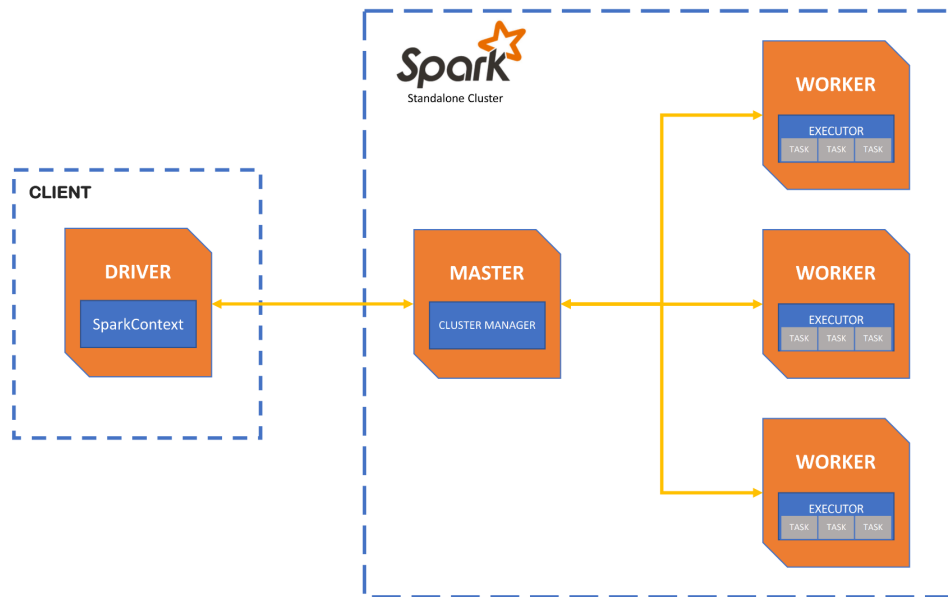
- GraphX is a component for graph processing and computation. It provides:
 - i. APIs for graph processing and building graph-parallel computation.
 - ii. Optimized in-memory representation of graphs.
 - iii. Pre-built algorithms like PageRank, connected components, and triangle counting.

6. PySpark

- PySpark is the Python API for Spark. It includes:
 - i. Full access to Spark's APIs in Python.
 - ii. Ability to use Python libraries in Spark jobs.
 - iii. Support for interactive data analysis using Jupyter Notebooks.
-

Cluster in Apache Spark

- In the context of Apache Spark, a cluster refers to a collection of interconnected computers (nodes) that work together to perform distributed data processing tasks. A Spark cluster is designed to handle large-scale data processing by distributing computation across multiple machines.



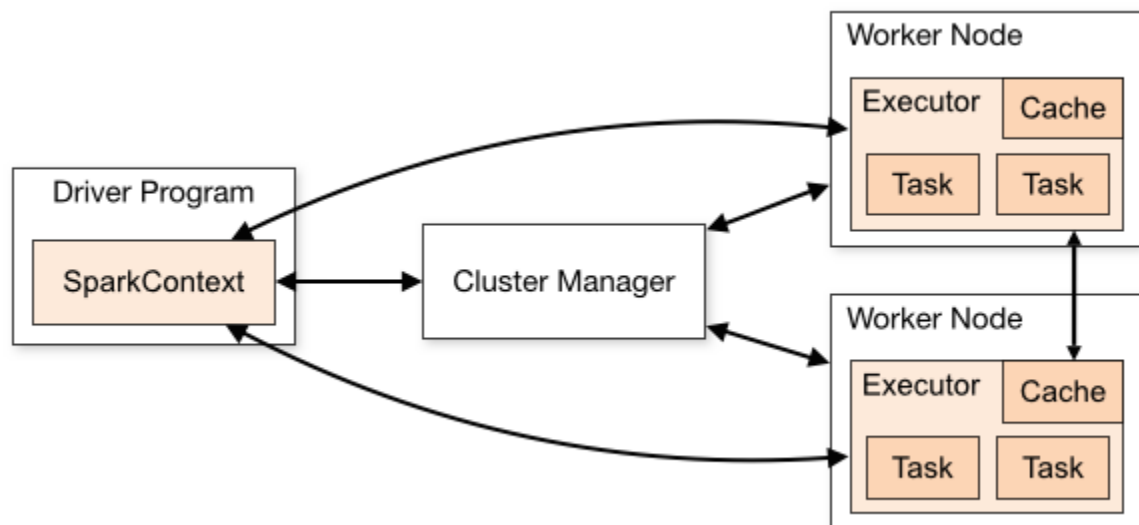
- Key Components of a Spark Cluster
 1. Master Node:
 - Description: The central coordinating node that manages the cluster resources and job scheduling.
 - Responsibilities:
 - Receives job submissions from clients.
 - Divides the job into tasks and distributes them to worker nodes.
 - Monitors the execution of tasks and handles fault tolerance.
 2. Worker Nodes (Executor Nodes):
 - Description: The nodes where actual data processing tasks are executed.
 - Responsibilities:
 - Execute the tasks assigned by the master node.
 - Store data in memory or disk as required.
 - Communicate task status and progress back to the master node.
 3. Cluster Manager:
 - Description: The component responsible for resource management and scheduling in the cluster.
 - Examples:

- Standalone Mode: Simple built-in cluster manager.
- Apache Hadoop YARN: Resource management platform for Hadoop clusters.
- Apache Mesos: General-purpose cluster manager.
- Kubernetes: Container orchestration system for automating deployment and management.

4. Client:

- Description: The interface through which users interact with the Spark cluster.
- Responsibilities:
 - Submits applications to the cluster.
 - Can be a Spark shell, notebook, or any other application submitting jobs.

Apache Spark Architecture



- Apache Spark's architecture is designed to efficiently handle large-scale data processing tasks across distributed computing clusters.
- Here's an overview of the key components and layers that constitute Spark's architecture:
 - a. Cluster Manager:
 - Think of this as a traffic controller for Spark applications. It manages resources like memory and CPU across all the computers (nodes) in the cluster.

- b. Driver Program:
 - This is like the boss of a Spark application. It tells the cluster manager what needs to be done and coordinates the execution of tasks.
- c. Executors:
 - These are like workers in the factory. They carry out the actual computations. Each executor runs on a node in the cluster and manages data stored on that node.
- d. RDD (Resilient Distributed Dataset):
 - RDD is how Spark handles data. It's a collection of data items that can be processed in parallel across the cluster. RDDs are fault-tolerant, meaning they can recover automatically if there's a failure.
- e. Jobs, Stages, and Tasks:
 - Jobs: These are the high-level instructions given by the driver program. A job typically involves transforming or analyzing data.
 - Stages: Jobs are broken down into stages. Each stage represents a set of tasks that can be executed together.
 - Tasks: These are the smallest units of work in Spark. Each task performs a computation on a piece of data.
- f. Spark Context (sc):
 - This is like a gateway to Spark. It connects your Spark program to the cluster and manages resources.

Component	Description	Analogy
Cluster Manager	Manages resources across the Spark cluster, like memory and CPU.	Traffic Controller
Driver Program	Coordinates the execution of Spark jobs and communicates with the Cluster Manager.	Boss or Project Manager
Executors	Worker nodes in the Spark cluster that execute tasks and manage data on each node.	Workers in a Factory
RDD (Resilient Distributed Dataset)	Distributed collection of data items that can be operated on in parallel.	Containers holding pieces of a puzzle
Jobs	High-level instructions given by the Driver Program to process data (transformations or actions).	Major Tasks or Projects
Stages	Subdivisions of Jobs based on data dependencies, broken down into tasks.	Phases or Steps within a Project
Tasks	Smallest unit of work in Spark, performing computations on data items within an Executor.	Individual Actions or Steps
Spark Context (sc)	Entry point for Spark functionality, connecting the Spark application to the Cluster Manager.	Gateway or Interface to the Cluster

Explain In Detailed:

1. The Driver Program is nothing but a master.
2. When the file is launched it first goes to the driver program.
3. Driver program will accept it and connect to cluster manager
4. Cluster is nothing but a group of nodes.
5. To handle the cluster we have a cluster manager.
6. Cluster manager has information of the file where the file is and on which work node the file is loaded.
7. Cluster manager also has information which worker is free and worker assigned
8. As the cluster manager shares the details of the worker node with the driver program, the driver program will go directly to the worker node.
9. Driver program will go directly to the worker node.
10. Driver program shares the file per the execution to the executor.
11. Executor divides job into subtasks & this task will be parallel executor
12. After the task is done it is sent back to the executor & executor sent back to the program.

Sequence of Apache Spark Architecture and Processing:

1. Submission of Spark Application:
 - The Spark application is submitted, typically through a Spark-submit script or programmatically using Spark APIs.
2. Initialization of Driver Program:
 - The submission starts with initializing the Driver Program.
 - The Driver Program is the main entry point that coordinates the execution of the Spark application.
3. Connection to Cluster Manager:
 - The Driver Program connects to the Cluster Manager (e.g., YARN, Mesos, Kubernetes) to request resources for executing tasks.
4. Resource Allocation (Executors):
 - The Cluster Manager allocates resources (Executors) based on the requirements specified by the Driver Program (e.g., number of CPUs, memory).
5. Task Distribution:
 - The Driver Program breaks down the Spark application into tasks and sends these tasks to the allocated Executors.
 - Tasks are divided into stages based on the dependencies between RDDs (Resilient Distributed Datasets).
6. Executor Execution:
 - Executors execute tasks in parallel on the data partitions (RDDs) stored across the cluster.
 - Each Executor manages and processes data locally on its assigned node.
7. Data Processing (RDD Operations):

- Executors perform data transformations and actions on RDDs as specified by the Spark application.
 - Operations include map, filter, reduce, join, and aggregate, among others.
8. Result Aggregation:
- Executors aggregate results of computations and transformations on RDDs.
 - Intermediate results are stored in memory (or optionally on disk) for efficient processing.
9. Data Shuffling (if required):
- During operations like reduceByKey or groupByKey, data may need to be shuffled across Executors to aggregate results.
 - Spark optimizes shuffling through intelligent partitioning and scheduling.
10. Result Collection:
- After all tasks are executed, results from Executors are collected by the Driver Program.
 - The Driver Program aggregates and processes final results as needed (e.g., saving to external storage, displaying results).
11. Application Completion:
- Once all tasks and stages are completed, the Spark application finishes execution.
 - Executors are released, and resources are returned to the Cluster Manager.
-

Transformations & Actions

Transformations

- Transformations are operations applied to an RDD (Resilient Distributed Dataset), DataFrame, or Dataset that return a new RDD, DataFrame, or Dataset, respectively.
- Transformations are lazy, meaning they don't immediately compute their results.
- Instead, they build up a lineage of transformations to be applied when an action is executed. This laziness allows Spark to optimize the processing pipeline.
- Here are some common transformation functions:
 1. map(): Applies a function to each item in the input RDD and returns a new RDD.
 2. filter(): Returns a new RDD by selecting only the elements of the original RDD that pass the test put forth by the filter function.
 3. flatMap(): Similar to map, but each input item can be mapped to multiple output items.
 4. union(): Returns a new RDD that contains the union of the elements in the source RDD and another RDD.
- There are two types of transformations in PySpark:

1. Narrow Transformation:

- These transformations do not require the data to be shuffled across partitions.
 - Examples `map()`, `mapPartition()`, `flatMap()`, `filter()`, and `union()`.
2. Wide Transformation:
- These transformations require the data to be shuffled across partitions.
 - Examples `groupByKey()`, `reduceByKey()`, `aggregateByKey()`, `join()`, and `repartition()`.

Actions

- In PySpark, actions are operations that return a value to the driver program. Unlike transformations, which produce RDDs (Resilient Distributed Datasets), DataFrames, or DataSets, actions produce a computed value.
- Example, an action function such as `count()` will produce a result back to the Spark driver while a `collect()` transformation function will not.
- Actions trigger the computation of any previously constructed PySpark transformations to be initialized. Since transformations are lazily initialized, any call to a Spark action will result in these data abstractions in the Spark directed acyclic graph (DAG) to be evaluated.
- Example of Action functions:
 1. `reduce()`: Aggregates the elements of the dataset using the specified binary operator1.
 2. `collect()`: Returns the complete dataset as an array1.
 3. `count()`: Returns the count of elements in the dataset1.
 4. `first()`: Returns the first element of the dataset1.
 5. `take()`: Returns the first n elements of the dataset1.

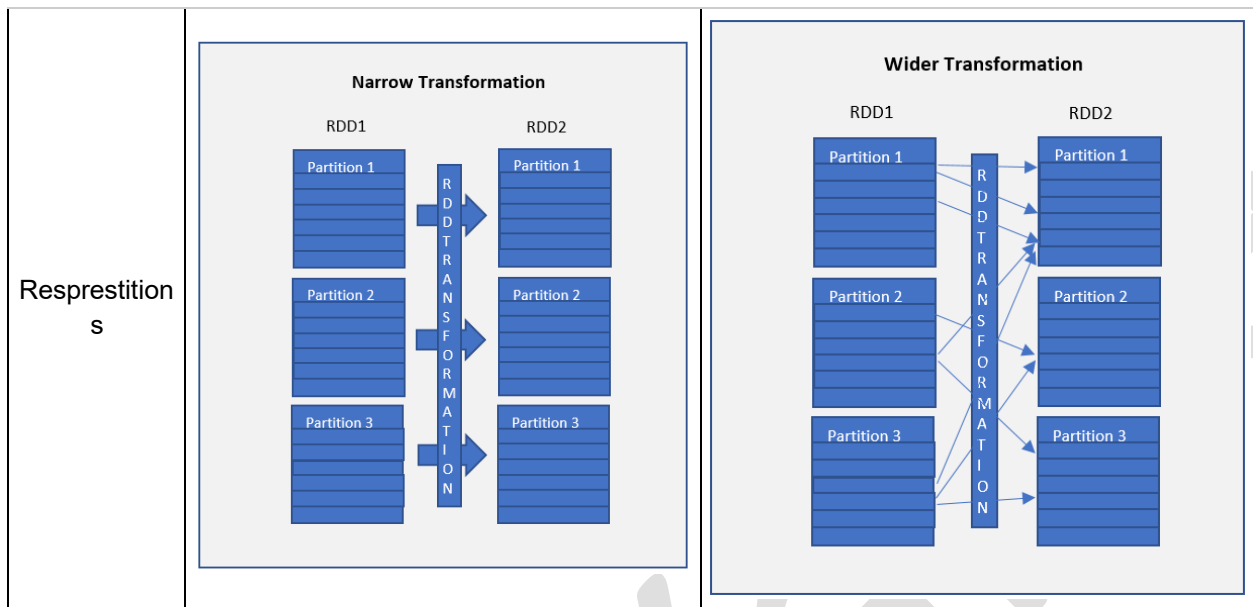
Transformations VS Actions

Feature	Transformations	Actions
Definition	Operations that define a new RDD, DataFrame, or Dataset from an existing one.	Operations that trigger the execution of the computation graph.
Execution	Lazy (delayed execution until an action is called).	Eager (immediately trigger the execution of the transformations).
Output	Returns a new RDD, DataFrame, or Dataset.	Returns a result to the driver program or writes data to storage.
Optimization	Build up a logical execution plan.	Trigger the execution, leading to the creation of a physical plan.
Data Shuffling	Some transformations may involve	Actions may cause transformations to shuffle

	shuffling data (wide transformations).	data if necessary.
Common Operations	map, filter, flatMap, mapPartitions, union, intersection, distinct, groupByKey, reduceByKey, sortByKey, join, cogroup.	collect, count, first, take, takeSample, reduce, aggregate, fold, saveAsTextFile, saveAsSequenceFile, saveAsObjectFile, saveAsHadoopFile, foreach.
Purpose	To define the data processing pipeline.	To get the result of the data processing pipeline or save the result.
Function	Typically used to build up a computation chain.	Typically used to retrieve or store the result of a computation chain.

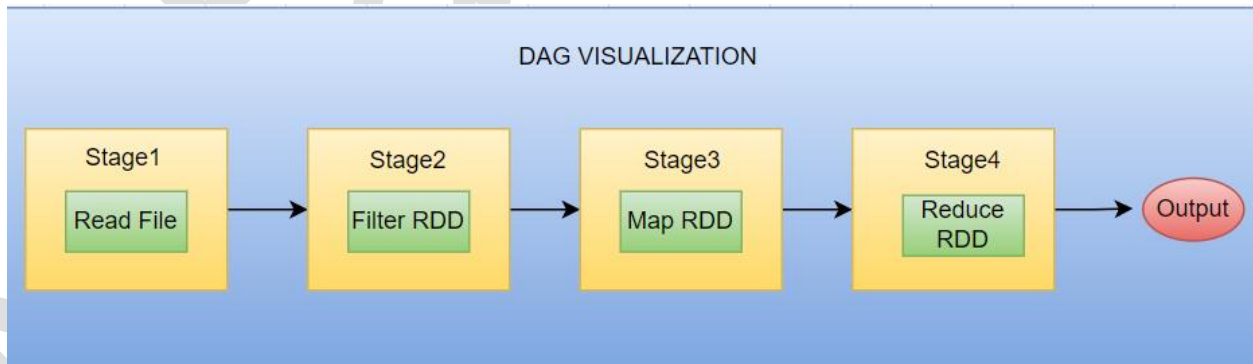
Narrow Transformations vs Wide Transformations

Feature	Narrow Transformations	Wide Transformations
Definition	Transformations where each input partition contributes to only one output partition.	Transformations where each input partition contributes to multiple output partitions.
Data Shuffling	No data shuffling required.	Requires data shuffling across the network.
Efficiency	More efficient due to lack of shuffling.	Less efficient due to the overhead of shuffling.
Fault Tolerance	Easier to recompute since partitions are not dependent on others.	More complex to recompute due to data shuffling and partition dependencies.
Examples	map, filter, flatMap, mapPartitions, union, sample, coalesce	groupByKey, reduceByKey, aggregateByKey, sortByKey, join, cogroup, repartition
Use Case	Applied when transformations can be performed independently on partitions.	Applied when transformations require data to be grouped or combined across partitions.
Partition Contribution	Each partition of the input RDD contributes to a single partition of the output RDD.	Each partition of the input RDD can contribute to multiple partitions of the output RDD.



Directed Acyclic Graph (DAG)

- In Apache Spark, DAG stands for Directed Acyclic Graph. It's a fundamental concept used by Spark's execution engine to represent and optimize the flow of operations in a data processing job.
- Here's a simple explanation
 1. Directed: The operations are executed in a specific order.
 2. Acyclic: There are no loops or cycles in the execution plan. This means that each stage depends on the completion of the previous stage, and each task within a stage can run independently of the other.

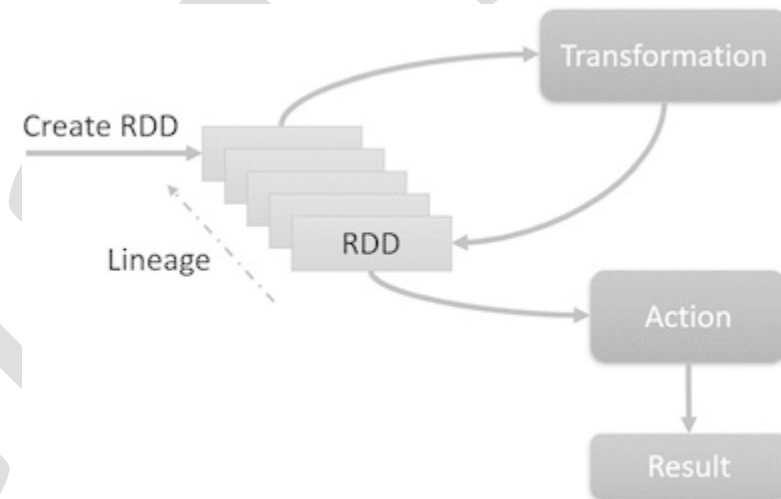


- At a high level, a DAG represents the logical execution plan of a Spark job. When a Spark application is submitted, Spark translates the high-level operations (such as transformations and actions) specified in the application code into a DAG of stages and tasks.

- The need for DAG in Spark arises from the fact that Spark is a distributed computing framework, which means it is designed to run on a cluster of machines. To effectively execute a Spark job across a cluster, Spark needs to break down the job into smaller, independent tasks that can be executed in parallel across the machines.
- The DAG plays a critical role in this process by providing a logical execution plan for the job. The DAG breaks the job down into a sequence of stages, where each stage represents a group of tasks that can be executed independently of each other. The tasks within each stage can be executed in parallel across the machines.
- The DAG allows Spark to perform various optimizations, such as pipelining, task reordering, and pruning unnecessary operations, to improve the efficiency of the job execution. By breaking down the job into smaller stages and tasks, Spark can execute them in parallel and distribute them across a cluster of machines for faster processing.

Lazy Evaluation

- Lazy evaluation is a core feature in Apache Spark, allowing it to optimize and streamline the execution of data transformations.
- Transformations are not executed immediately but are deferred until an action triggers their execution. This helps Spark optimize the computation process and improve performance.



- Key Concepts

1. Transformations:

- Operations that define a new RDD, DataFrame, or Dataset from an existing one. These are lazy and only define the transformation steps without immediately executing them.
- Examples: map, filter, flatMap

2. Actions:

- Operations that trigger the execution of all the recorded transformations and return a result to the driver program or write data to external storage.

- Examples: collect, count, saveAsTextFile
 - Benefits of Lazy Evaluation
 1. Optimization:
 - Spark can analyze the complete set of transformations and optimize the execution plan.
 - Minimizes data shuffling and merges multiple transformations into a single pass.
 2. Efficiency:
 - Prevents redundant data processing.
 - Reduces computational overhead by collapsing multiple transformations into a single pass over the data.
 3. Fault Tolerance:
 - The lineage graph helps Spark recover from failures.
 - If a node fails, Spark can recompute lost data using the lineage information.
-

Resilient Distributed Dataset (RDD)

- An RDD, or Resilient Distributed Dataset, is the primary data structure of Apache Spark. It's like a big collection of data that's spread out across many computers so Spark can work on lots of pieces of the data at the same time, which makes things go faster.
- Here's what makes RDDs special:
 1. **Resilient:** They can recover quickly from problems because the data is replicated across different computers.
 2. **Distributed:** The data is split up and stored on multiple nodes in a cluster, allowing parallel processing.
 3. **Dataset:** It's a collection of data items that Spark can work with.
- Key Characteristics of RDDs

Characteristic	Description	Benefit
Immutability	RDDs cannot be changed once created.	Simplifies reasoning about data and enhances fault tolerance.
Distributed	Data is distributed across cluster nodes.	Enables parallel processing and scalability.
Fault Tolerance	Can recover from node failures using lineage.	Ensures reliable data processing.
Lazy Evaluation	Transformations are not executed until an action is called.	Allows for optimization of execution plans.
Lineage Information	Metadata about transformations is maintained.	Provides fault tolerance and helps in data recomputation.

Partitioning	RDDs are divided into partitions.	Enhances parallel processing and efficiency.
Persistence	RDDs can be cached in memory or on disk.	Improves performance for iterative algorithms.
Transformations	Create new RDDs from existing ones, and are lazy.	Allow for flexible and optimized data manipulation.
Actions	Trigger execution of transformations, are eager.	Return results to the driver or write data to external storage.
In-Memory Computing	Leverages in-memory processing for speed.	Significantly faster data processing compared to disk-based systems.
Coarse-Grained Ops	Operations apply to the entire dataset.	Simplifies distributed data processing.
Data Locality	Processes data where it is stored.	Reduces network I/O and enhances performance.

- Creating RDDs
 1. **Parallelized Collections:** Create an RDD from a Python collection.
 2. **External Datasets:** Create an RDD by loading data from external storage.
 3. **Transforming Existing RDDs:** Create new RDDs by applying transformations to existing ones.
 4. **Creating Pair RDDs:** Create RDDs containing key-value pairs for operations like aggregation.

1. Parallelized Collections

- You can create an RDD from an existing Python collection (e.g., list or array) using the `parallelize` method.

```
from pyspark import SparkContext

# Initialize a SparkContext
sc = SparkContext("local", "RDD Example")

# Create an RDD from a Python list
data = [1, 2, 3, 4, 5]
rdd = sc.parallelize(data)

# Print the elements of the RDD
print(rdd.collect())
```

2. External Datasets

- You can create an RDD by loading data from external storage systems like HDFS, S3, or the local file system using the `textFile` method.

```
from pyspark import SparkContext

# Initialize a SparkContext
sc = SparkContext("local", "RDD Example")

# Create an RDD from an external text file
rdd = sc.textFile("path/to/file.txt")

# Print the first few lines of the RDD
print(rdd.take(5))
```

3. Transforming Existing RDDs

- You can create new RDDs by applying transformations to existing RDDs

```
from pyspark import SparkContext

# Initialize a SparkContext
sc = SparkContext("local", "RDD Example")

# Create an RDD from a Python list
data = [1, 2, 3, 4, 5]
rdd = sc.parallelize(data)

# Apply a map transformation to create a new RDD
rdd2 = rdd.map(lambda x: x * 2)

# Apply a filter transformation to create another RDD
rdd3 = rdd2.filter(lambda x: x > 5)

# Print the elements of the new RDDs
print(rdd2.collect()) # [2, 4, 6, 8, 10]
print(rdd3.collect()) # [6, 8, 10]
```

4. Creating Pair RDDs

- Pair RDDs contain key-value pairs and can be created by transforming regular RDDs.

```
from pyspark import SparkContext
```

```
# Initialize a SparkContext
sc = SparkContext("local", "RDD Example")

# Create an RDD from a list of tuples (key-value pairs)
data = [("a", 1), ("b", 2), ("c", 3)]
pair_rdd = sc.parallelize(data)

# Apply a reduceByKey transformation to sum values by key
reduced_rdd = pair_rdd.reduceByKey(lambda a, b: a + b)

# Print the elements of the pair RDD
print(reduced_rdd.collect()) # [('a', 1), ('b', 2), ('c', 3)]
```

RDD vs Dataframe vs Dataset

Feature	RDD	DataFrame	Dataset
Level of Abstraction	Low-level (fine-grained control)	High-level (SQL-like, relational)	High-level with type safety
Type Safety	No	No (dynamic typing)	Yes (compile-time type safety)
API	Low-level API	High-level API, supports SQL queries	High-level API with type safety
Performance	Generally slower	Faster due to Catalyst and Tungsten	Similar to DataFrames, with type safety
Optimizations	Manual optimizations needed	Optimized by Catalyst and Tungsten	Optimized by Catalyst and Tungsten
Fault Tolerance	Lineage-based recovery	Lineage-based recovery	Lineage-based recovery
Serialization	Manual (Java/Kryo)	Internally optimized	Internally optimized with encoders
Use Case	Unstructured/semi-structured data	Structured data with SQL-like queries	Structured data with type safety

Spark context vs Spark Session

Feature	SparkContext	SparkSession
Introduction	Spark 1.x	Spark 2.0+
Functionality	Core Spark functions (RDDs)	Unified interface for DataFrame, Dataset, SQL, and core Spark functions
API Scope	Limited to low-level API	High-level API with support for DataFrames and Datasets
Initialization	Requires manual configuration for SQL/HiveContext	Encapsulates SparkContext, SQLContext, HiveContext
Ease of Use	More complex for structured data	Simplified, more user-friendly API for all data types
Code Example	<pre>from pyspark import SparkContext # Initialize SparkContext sc = SparkContext("local", "SparkContext Example") # Create an RDD from a list data = [1, 2, 3, 4, 5] rdd = sc.parallelize(data) # Perform an action print(rdd.collect())</pre>	<pre>from pyspark.sql import SparkSession # Initialize SparkSession spark = SparkSession.builder \ .appName("SparkSession Example") \ .master("local") \ .getOrCreate() # Create a DataFrame from a list data = [(1,), (2,), (3,), (4,), (5,)] df = spark.createDataFrame(data, ["number"]) # Perform an action df.show()</pre>

Spark Application, Job, Stage, Tasks

Understanding the hierarchy of Spark's execution model is key to effectively managing and optimizing Spark applications. Here's a breakdown of the key concepts: Spark Application, Job, Stage, and Tasks.

1. Spark Application

- Definition: A Spark Application is the highest-level unit of computation in Spark. It includes the user program that runs on the cluster, which consists of the driver program and the executors on the cluster nodes.

- This is the whole program you write to process your data. It's like a factory where everything happens. When you run your code, Spark starts up an application which then gets divided into smaller pieces.
- Driver Program: The main program that controls the flow of the application and creates the SparkContext.
- Executors: The worker processes that run individual tasks.
- Components:
 - SparkContext: The entry point for the application, responsible for initializing the Spark application.
 - Driver: Manages the lifecycle of the Spark application, including scheduling, distribution, and monitoring of tasks.

2. Job

- Definition: A job is triggered by an action (e.g., `collect()`, `saveAsTextFile()`) in a Spark application. Each action results in a separate job.
- When you ask Spark to do something that produces an output, like counting the number of items in a dataset or saving it to a file, that's a job. It's like a project in the factory.
- Characteristics:
 - Jobs: Correspond to a high-level action in the Spark code.
 - Triggered By: Actions such as `collect`, `count`, `saveAsTextFile`, etc.

3. Stage

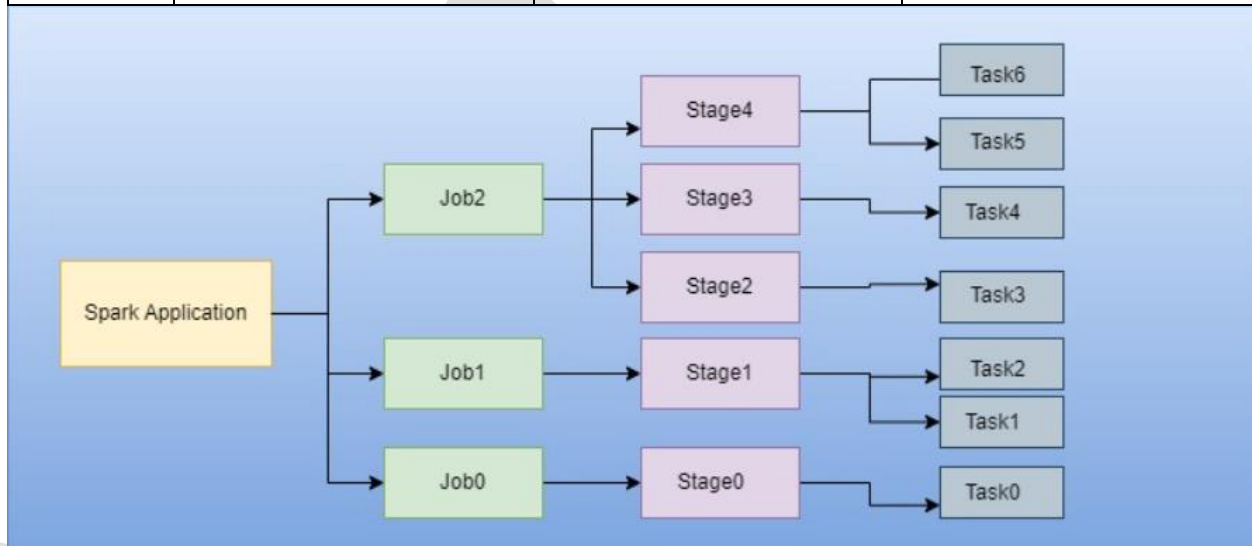
- Definition: Each job is divided into stages. Stages represent a set of parallel tasks that correspond to a sequence of transformations.
- Each job gets broken down into stages. A stage is a bunch of tasks that can run at the same time because they don't depend on each other. It's like breaking down the project into different departments in the factory.
- Characteristics:
 - Stages: Divided based on shuffling boundaries, i.e., when data needs to be reorganized (e.g., `reduceByKey`).
 - Dependencies: Stages are dependent on the completion of previous stages.
- Types of Stages:
 - Shuffle Map Stage: Involves shuffle operations.
 - Result Stage: Final stage, which performs an action to produce the result.
 - .

4. Tasks

- Definition: Tasks are the smallest unit of work in Spark. Each stage is divided into tasks that are distributed across the cluster nodes.

- These are the smallest units of work and they happen on individual pieces of data. Each task is done by one computer in your Spark cluster. It's like each worker in the department doing their part of the job.
- Characteristics:
 - Tasks: Execute the same code (like a function) on a partition of data.
 - Parallel Execution: Tasks run in parallel on different executors.
 - Granularity: Each task processes a partition of the data.
- Task Types:
 - Result Tasks: Fetch the final result of a stage.
 - Shuffle Tasks: Read and write intermediate data during shuffles.

Component	Description	Triggered By	Example
Spark Application	The highest-level unit of computation, includes driver and executors.	Initialized by creating SparkSession or SparkContext.	<code>spark = SparkSession.builder.appName("Example").getOrCreate()</code>
Job	Triggered by an action and comprises multiple stages.	Actions like <code>collect</code> , <code>saveAsTextFile</code> .	<code>result = rdd.collect()</code>
Stage	Subdivision of a job, divided by shuffle boundaries.	Transformed RDDs and dependencies.	<code>rdd2 = rdd.map(lambda x: x * 2)</code>
Task	The smallest unit of work, executed on partitions of data.	Divided from stages, run on executors.	Automatically managed by Spark.



- Detailed Execution Flow
 1. Spark Application starts by initializing a SparkSession or SparkContext.
 2. Job is triggered when an action is called.
 3. Stages are created within the job based on shuffle boundaries.
 4. Tasks are created from stages and executed on the cluster nodes.

Repartition vs Coalesce

Feature	Repartition	Coalesce
Shuffling	Yes, full shuffle	No, avoids full shuffle
Use Case	Increase or decrease partitions	Decrease partitions
Performance	More expensive due to shuffle	More efficient as it merges partitions
Flexibility	Can increase or decrease partitions	Typically used to decrease partitions
Syntax	<code>df.repartition(numPartitions)</code>	<code>df.coalesce(numPartitions)</code>
Example Use Case	Distributing data evenly across nodes	Reducing partitions after an operation to optimize
Typical Scenario	After a filter or join operation	Before a write operation to minimize small files

- Practical Considerations
 - **Repartition:** Use when you need to distribute data evenly across the cluster nodes or when increasing the number of partitions.
 - **Coalesce:** Use when you need to decrease the number of partitions without a shuffle, such as before writing out a DataFrame to avoid creating many small output files.
-

Broadcast Variable And Accumulator Variable

- Broadcast Variables
 - Broadcast variables are used to efficiently share large read-only data across all worker nodes. They are used when you have a dataset that needs to be accessed by all tasks in a Spark job, such as a large lookup table or a configuration object.
 - Characteristics:
 - Immutable: Once a broadcast variable is created, it cannot be modified. It is read-only.
 - Efficient Distribution: Spark sends a copy of the broadcast variable to each node once, rather than sending it with every task. This reduces communication overhead and speeds up the job.
 - Usage: Broadcast variables are used when you want to share a large dataset or configuration across multiple tasks without having to send it multiple times.
- Example:

```

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("example").getOrCreate()

# Example data to broadcast
broadcast_var = spark.sparkContext.broadcast([1, 2, 3])

# Accessing the broadcast variable in a transformation
def process_data(x):
    return x + sum(broadcast_var.value)

rdd = spark.sparkContext.parallelize([10, 20, 30])
result = rdd.map(process_data).collect()

print(result) # Output: [16, 26, 36]

```

- Accumulators

- Accumulators are variables that are used to accumulate sums or counts across tasks and can be used to implement counters or aggregations. They are primarily used for debugging and monitoring purposes, as they can be updated across multiple tasks.
- Characteristics:
 - Mutable: Accumulators are designed to be updated, typically in an additive manner.
 - Fault-Tolerant: Spark ensures that updates to accumulators are correctly handled even if tasks are retried, though their use with certain transformations (e.g., filter) can lead to inconsistencies.
 - Limited Visibility: Accumulators are only updated on the driver program, and their values are not guaranteed to be updated if an action (e.g., collect, count) is not performed.
- Example:

```

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("example").getOrCreate()

# Create an accumulator
accumulator = spark.sparkContext.accumulator(0)

# Update accumulator within a transformation
def add_to_accumulator(x):
    global accumulator
    accumulator.add(x)
    return x

```

```
rdd = spark.sparkContext.parallelize([10, 20, 30])
rdd.foreach(add_to_accumulator)
```

```
# Retrieve the accumulator value
print(accumulator.value) # Output: 60
```

Feature	Broadcast Variable	Accumulator Variable
Purpose	Efficiently share large read-only data	Accumulate sums or counts across tasks
Mutability	Immutable (read-only)	Mutable (can be updated)
Distribution	Distributed once to each node	Updated on the driver program
Usage	Sharing lookup tables, configuration	Implementing counters or aggregations
Fault Tolerance	Handled by Spark (efficient distribution)	Handled with retries but not guaranteed consistent updates

Join Strategies in Spark

- Joining data is a common operation in data processing, and Apache Spark provides multiple join strategies to efficiently handle different scenarios. Choosing the right join strategy is crucial for performance optimization, especially when working with large datasets. Here's an overview of the primary join strategies in Spark:

1. Broadcast Hash Join

- Definition: The smaller DataFrame/RDD is broadcasted to all worker nodes, allowing the larger DataFrame/RDD to be joined locally on each node.
- Spark will use this strategy if one side of the join is small enough to fit into memory (the broadcast threshold can be adjusted). This is often the fastest join because it avoids shuffling large amounts of data.
- Use Case: Ideal when one of the DataFrames/RDDs is small enough to fit in memory on each executor node.
- Performance: Very efficient for small datasets as it avoids shuffling.

2. Sort-Merge Join

- Definition: Both DataFrames/RDDs are sorted by the join key and then merged. This is the default join strategy in Spark when the data is large.
- This is the default strategy when neither side of the join is small enough for broadcasting and when `spark.sql.join.preferSortMergeJoin` is set to true. It's

generally good for large datasets because it's more scalable than shuffle hash join.

- Use Case: Used for large datasets when no dataset is small enough for a broadcast join.
- Performance: Can be expensive due to sorting and shuffling.
- Syntax: Automatically chosen by Spark when joining large datasets without explicit broadcast.

3. Shuffle Hash Join

- Definition: The larger DataFrame/RDD is partitioned and then a hash join is performed within each partition.
- If the broadcast hash join is not possible and if the Spark configuration parameter spark.sql.join.preferSortMergeJoin is set to false, Spark may use shuffle hash join. This strategy is good when both datasets are large but one is significantly smaller than the other.
- Use Case: Suitable when both DataFrames/RDDs are large but have an even distribution of keys.
- Performance: Can be more efficient than sort-merge join if the hash function distributes data evenly.
- Syntax: Typically chosen by Spark based on internal heuristics.

4. Broadcast Nested Loop Join

- Definition: A Cartesian join that evaluates all pairs of elements from both DataFrames/RDDs, used when no join keys are specified.
- Broadcast: The smaller dataset is sent to every node in the cluster so that it's available in memory for quick access.
Nested Loop: For each row in the larger dataset, Spark will iterate over every row in the broadcasted smaller dataset to find matching rows based on the join condition.
- Use Case: Used for very small datasets or when a Cartesian product is required.
- Performance: Inefficient for large datasets due to the Cartesian product.

5. Cartesian Product

- Definition: Evaluates all pairs of elements from both DataFrames/RDDs, similar to the broadcast nested loop join but typically used for much smaller datasets or specific requirements.
- Spark will only use this if you explicitly ask for a Cartesian product, which is a join without any join condition. It's very expensive in terms of computation and should be used sparingly.
- Use Case: Used when a Cartesian product is required.
- Performance: Very inefficient for large datasets as it results in $O(n*m)$ pairs.
- Shuffling: No, but very costly in terms of computational complexity.

Join Strategy	Use Case	Performance	Shuffling	Syntax
---------------	----------	-------------	-----------	--------

Broadcast Hash Join	Small DataFrame/RDD joined with large	Very efficient for small datasets	No	<code>df_large.join(F.broadcast(df_small), "key")</code>
Sort-Merge Join	Large datasets with sortable keys	Expensive due to sorting and shuffling	Yes	<code>df1.join(df2, "key")</code>
Shuffle Hash Join	Large datasets with even key distribution	Efficient if hash function is good	Yes	Automatically chosen by Spark
Broadcast Nested Loop Join	Small datasets or Cartesian product	Inefficient for large datasets	No	<code>df1.crossJoin(df2)</code>
Cartesian Product	Cartesian product required	Very inefficient for large datasets	No	<code>df1.crossJoin(df2)</code>

Practical Considerations

- **Broadcast Hash Join:** Use when one dataset is small. It avoids the shuffle and is very efficient.
- **Sort-Merge Join:** Default for large datasets. Ensure data is partitioned and sorted properly to optimize performance.
- **Shuffle Hash Join:** Automatically chosen by Spark based on heuristics. Monitor and optimize if necessary.
- **Broadcast Nested Loop Join:** Avoid for large datasets. Use for small datasets or specific Cartesian product requirements.
- **Cartesian Product:** Inefficient and should be avoided for large datasets unless absolutely necessary.

Shuffling In Spark

- Shuffling is the process of redistributing data across partitions or nodes in a cluster to perform operations that require data to be reorganized. This process involves moving data between different partitions or nodes to satisfy dependencies between stages in a Spark job.
- When Does Shuffling Occur?:
- Shuffling typically occurs during operations that require data to be reorganized, such as:
 - `reduceByKey`: Combines values with the same key across partitions.
 - `groupByKey`: Groups values with the same key across partitions.
 - `join`: Merges datasets based on a common key.
 - `distinct`: Removes duplicate values.
 - `coalesce` and `repartition`: Changes the number of partitions.
- How Shuffling Works:
 - Data Partitioning: Data is distributed across partitions on different nodes.

- Shuffle Read: Data is read from different partitions or nodes, involving network I/O.
- Shuffle Write: Data is written to disk or memory, creating new partitions.
- Data Exchange: Data is exchanged between nodes, involving disk and network I/O.
- Performance Considerations:
 - Network I/O: Data transfer over the network can be slow.
 - Disk I/O: Writing and reading data from disk adds overhead.
 - Sorting and Grouping: Sorting or grouping data can be time-consuming.
- Optimizing Shuffling:
 - Reduce Shuffle Operations: Minimize the number of shuffles by optimizing transformations.
 - Use Appropriate Partitioning: Ensure even distribution of data across partitions.
 - Tune Spark Configuration: Adjust configurations like `spark.sql.shuffle.partitions` to optimize partitioning.
 - Persist Intermediate Data: Use `persist()` or `cache()` to store intermediate data in memory.
- Example:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("example").getOrCreate()

# Example DataFrame
data = [("A", 1), ("B", 2), ("A", 3), ("B", 4)]
df = spark.createDataFrame(data, ["key", "value"])

# Perform groupBy and aggregate
result = df.groupBy("key").avg("value")

# Show the result
result.show()
```

Driver Out of Memory in Spark

- Running out of memory on the driver node in Apache Spark can cause your application to fail. The driver node is responsible for orchestrating the entire Spark application, including managing the `SparkContext`, scheduling tasks, and maintaining metadata

about the RDDs/DataFrames. Here are common causes and solutions to address driver memory issues.

Cause	Description	Impact	Solution
Insufficient Driver Memory	The memory allocated to the driver may not be enough for the workload.	Driver runs out of memory, causing application failure.	Increase driver memory allocation using the <code>--driver-memory</code> flag or Spark configuration.
Large Data Collections	Using actions like <code>collect()</code> that retrieve large amounts of data back to the driver.	Driver memory is overwhelmed by the large dataset.	Use distributed actions (<code>foreach</code> , <code>write</code>) instead of <code>collect()</code> .
Inefficient Caching	Caching large data sets without proper unpersisting can fill up the driver's memory.	Driver memory fills up, leading to potential memory overflow.	Unpersist datasets when they are no longer needed using <code>unpersist()</code> .
Large Broadcast Variables	Broadcasting large variables that exceed the available memory on the driver.	Excess memory consumption on the driver, leading to potential crashes.	Optimize and reduce the size of broadcast variables.
Memory Leaks	Poorly written user-defined functions (UDFs) or other code that doesn't release memory properly.	Gradual increase in memory usage, eventually leading to driver OOM.	Ensure proper memory management in UDFs and avoid holding references unnecessarily.
Excessive Accumulators	Using a large number of accumulators or storing large values in them.	Significant memory consumption by accumulators on the driver.	Limit the use of large accumulators and store only essential data.
Garbage Collection Overhead	Inefficient garbage collection can cause memory to be used faster than it is released.	Memory is not released efficiently, causing high memory usage.	Tune garbage collection parameters and monitor GC performance.
Improper Serialization	Using default Java serialization which is less efficient than Kryo serialization.	Higher memory usage due to less efficient serialization.	Use Kryo serialization for better efficiency by setting <code>spark.serializer</code> to <code>org.apache.spark.serializer.KryoSerializer</code> .
Resource Misconfiguration	Incorrectly configured Spark properties related to memory management.	Suboptimal memory usage and potential driver OOM.	Configure Spark properties correctly, such as <code>spark.driver.memory</code> .
Data Skew	Uneven distribution of data can lead to certain tasks using more memory than others.	Uneven memory usage, with some tasks consuming excessive memory.	Monitor data distribution and use techniques like salting to handle skew.

Executor Out of Memory (OOM)

- Executor out of memory occurs when the memory allocated to a Spark executor is exhausted, causing the executor to fail. This can lead to job failures, reduced performance, and the inability to process tasks effectively.

Cause	Description	Impact	Solution
Insufficient Executor Memory	The memory allocated to executors may not be enough for the workload.	Executor runs out of memory, causing task failures.	Increase executor memory allocation using the <code>--executor-memory</code> flag or Spark configuration.
Large Data Skew	Uneven distribution of data can lead to certain executors processing more data than others.	Some executors consume excessive memory, leading to OOM errors.	Monitor and handle data skew using techniques like salting and repartitioning.
High Shuffle Data Volume	Large amounts of data being shuffled between executors.	High memory consumption during shuffles, leading to potential OOM.	Increase shuffle partitions and use optimized shuffle settings.
Inefficient Caching	Caching large datasets without proper memory management.	Executors' memory fills up, leading to OOM errors.	Cache only necessary datasets and unpersist when no longer needed.
Large Join Operations	Performing large join operations that require significant memory.	High memory usage during join operations, leading to potential OOM.	Use broadcast joins for small tables and optimize join strategies.
Large Broadcast Variables	Broadcasting large variables that exceed available executor memory.	Excess memory consumption on executors, leading to OOM errors.	Optimize and reduce the size of broadcast variables.
Memory Leaks	Poorly written UDFs or other code that doesn't release memory properly.	Gradual increase in memory usage, eventually leading to executor OOM.	Ensure proper memory management in UDFs and avoid holding references unnecessarily.
Excessive Garbage Collection	Inefficient garbage collection can cause executors to run out of memory.	Executors spend too much time in GC, leading to OOM errors.	Tune garbage collection parameters and monitor GC performance.

Improper Serialization	Using inefficient serialization mechanisms.	Higher memory usage due to inefficient serialization.	Use efficient serialization like Kryo by setting spark.serializer to org.apache.spark.serializer.KryoSerializer.
Resource Misconfiguration	Incorrectly configured Spark properties related to memory management.	Suboptimal memory usage and potential executor OOM.	Configure Spark properties correctly, such as spark.executor.memory.

Executor Out of Memory vs. Driver Out of Memory

Aspect	Executor Out of Memory (OOM)	Driver Out of Memory (OOM)
Definition	The memory allocated to a Spark executor is exhausted.	The memory allocated to the Spark driver is exhausted.
Location	Occurs in executor JVMs running on worker nodes.	Occurs in the driver JVM running on the master node.
Common Causes	- Insufficient executor memory allocation	- Insufficient driver memory allocation
	- Large data skew	- Large data collections
	- High shuffle data volume	- Inefficient caching
	- Inefficient caching	- Large broadcast variables
	- Large join operations	- Memory leaks
	- Large broadcast variables	- Excessive accumulators
	- Memory leaks	- Garbage collection overhead
	- Excessive garbage collection	- Improper serialization
	- Improper serialization	- Resource misconfiguration
	- Resource misconfiguration	- Data skew
Impact	- Task failures and retries	- Driver crash leading to job failure
	- Performance degradation due to increased garbage collection and task retries	- Loss of job state and progress
	- Potential job failure if tasks cannot complete	

Solutions	- Increase executor memory allocation	- Increase driver memory allocation
	- Handle data skew with techniques like salting and repartitioning	- Avoid using collect() for large datasets
	- Optimize shuffle operations with more partitions and better settings	- Efficiently cache and unpersist datasets
	- Efficiently cache datasets and unpersist when not needed	- Reduce the size of broadcast variables
	- Use broadcast joins for small tables	- Avoid memory leaks in user-defined functions
	- Optimize garbage collection settings	- Limit the use of accumulators
	- Use efficient serialization like Kryo	- Tune garbage collection parameters
	- Correctly configure Spark properties	- Use efficient serialization like Kryo
Configuration Parameters	- spark.executor.memory	- spark.driver.memory
	- spark.executor.cores	- spark.driver.maxResultSize
	- spark.executor.extraJavaOptions for GC tuning	- spark.driver.extraJavaOptions for GC tuning
	- spark.sql.shuffle.partitions	- Proper resource configuration

Practical Example of

- Q. To process 25GB data in spark
 1. How many CPU cores are required?
 2. How many executors are required?
 3. How much of each executor memory is required?
 4. What is the total memory required?
-
1. HOW many executor CPU cores are required to process 25 GB data?
 - ----- Convert GB to MB
 - $256\text{GB} \times 250024 \text{ MB} = 25600 \text{ MB}$
 - ----- To get Partitions divided total MB by Partition Size (i.e 128 MB)
 - Number of Partitions - $25600 \text{ MB} / 128 \text{ MB} = 200$
 - ----- Final Answer
 - Number of CPU cores = Number of partitions = 200
 - *By default, Spark creates one partition for each block of the file (blocks being 128MB by default in HDFS). but you can also ask for a higher number of partitions by passing a larger value.*
 2. HOW many executors are required to process 25 GB data?

- Avg CPU cores for each executor = 4
 - ----- 4 CPU Core per Executor
 - Total number of executor = $200 / 4 = 50$
 - *To get better job performance in spark. researchers have found that we can take 2 to 5 maximum core for each executor*
3. How much memory is required to process 25 GB data?
- ----- As we have 4 CPU Cores Per executor
 - CPU cores for each executor = 4
 - Memory for each executor = $4 * 512 \text{ MB} = 2 \text{ GB}$
 - *Expected memory for each core = Minimum 4 x (default partition size) = $4 * 128 \text{ MB} = 512 \text{ MB}$*
4. What is the total memory required to process 25 GB data?
- Total number of executor = 50
 - Memory for each executor = 2 GB
 - Total Memory for all the executor = $50 * 2 \text{ GB} = 100 \text{ GB}$
 - *Executor memory is not less than 1.5 times of spark reserved memory (Single core executor memory should not be less than 450MB)*

Spark Submit

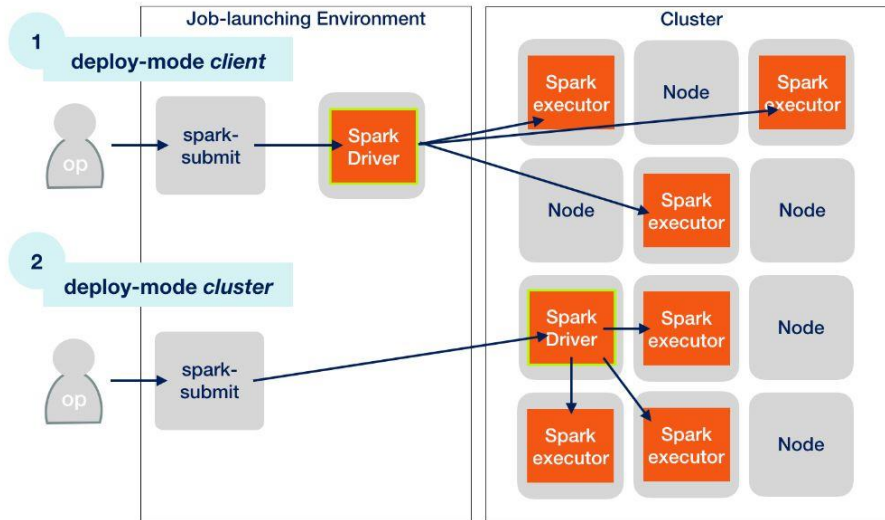
- Spark Submit is a powerful command-line tool provided by Apache Spark to submit applications to a Spark cluster for execution.
- It allows you to specify various configuration options and parameters to control how your Spark application is run on the cluster.

Option	Description	Example Usage
<code>--master <master-url></code>	Specifies the cluster manager to connect to (local, yarn, mesos, etc.)	<code>--master local[4]
--master yarn
--master spark://host:port</code>
<code>--deploy-mode <mode></code>	Specifies whether to deploy the driver on the local machine (client) or cluster (cluster)	<code>--deploy-mode client
--deploy-mode cluster</code>
<code>--class <main-class></code>	For Java/Scala applications, specifies the entry point (main class)	<code>--class com.example.Main</code>
<code>--name <name></code>	Sets the name of the application	<code>--name "My Spark Application"</code>
<code>--conf <key=value></code>	Sets a configuration property	<code>--conf spark.executor.memory=4g
--</code>

		conf spark.executor.cores=2
--jars <comma-separated-jars>	Specifies additional JAR files to include on the classpath	--jars /path/to/jar1,/path/to/jar2
--files <comma-separated-files>	Specifies additional files to distribute with the job	--files /path/to/file1,/path/to/file2
--py-files <comma-separated-files>	Specifies additional Python files to distribute with the job	--py-files /path/to/module1.py,/path/to/module2.py
--executor-memory <memory>	Sets the memory per executor (e.g., 4g, 2g)	--executor-memory 4g
--executor-cores <cores>	Sets the number of cores per executor	--executor-cores 2
--num-executors <num>	Sets the number of executors to launch	--num-executors 4
--driver-memory <memory>	Sets the memory for the driver process	--driver-memory 2g
--driver-cores <cores>	Sets the number of cores for the driver process	--driver-cores 1
--queue <queue-name>	Specifies the YARN queue to submit to	--queue my_queue
--archives <comma-separated-archives>	Specifies archives to be extracted and placed in the working directory	--archives /path/to/archive1,/path/to/archive2
--executor-cores <cores>	Sets the number of cores per executor	--executor-cores 4

Deploy Mode in Spark

- In Apache Spark, deploy-mode refers to a configuration parameter that specifies where the driver process runs and how the application is deployed in the cluster. This setting determines the location of the Spark driver and how the application communicates with the cluster's resources.



- Types of Deploy Modes

1. Client Mode:

- Description: In this mode, the driver runs on the machine that initiates the Spark job (typically where spark-submit is executed). The application communicates directly with the cluster's worker nodes to execute tasks.
- Usage: Preferred for interactive and debugging purposes as it allows easy access to the driver's logs and Spark UI.
- Command: `--deploy-mode client`

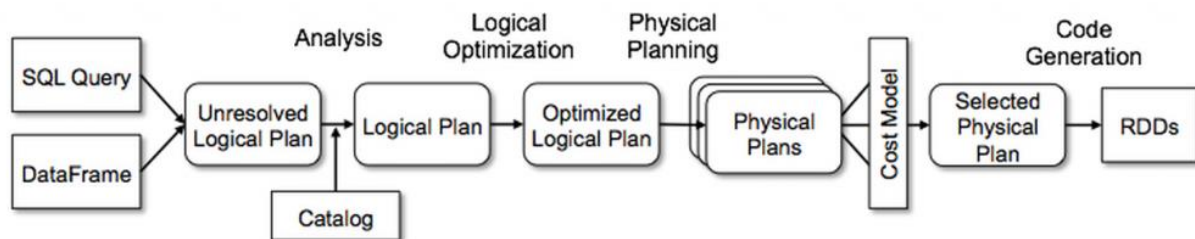
2. Cluster Mode:

- Description: In cluster mode, the driver runs within one of the nodes in the cluster. The client that launches the job does not need to stay active after submission, making it suitable for production deployments where the client machine may not have direct access to the cluster resources.
- Usage: Recommended for production deployments to ensure the application is not affected by client-side issues or network latency.
- Command: `--deploy-mode cluster`

Aspect	Client Mode	Cluster Mode
Driver Location	Runs on the machine where spark-submit is executed.	Runs on one of the worker nodes in the cluster.
Client Machine Dependency	Client machine must remain active during job execution.	Client machine can disconnect after job submission.
Suitability	Suitable for interactive development and debugging.	Suitable for production deployments.
Accessibility	Direct access to driver logs and	Limited direct access to driver logs

	Spark UI from the client.	and UI (usually accessible via cluster UI).
Network Dependency	Higher dependency on network latency and reliability.	Lower dependency on network due to driver locality.
Resource Utilization	Less efficient utilization of cluster resources for drivers.	More efficient utilization of cluster resources.
Job Management	Easier to manage the job lifecycle and debug on client side.	Job management handled within the cluster environment.
Security	May require additional security considerations for client access.	Simplifies security management by centralizing execution within the cluster.
Use Cases	Development, debugging, and short-lived applications.	Long-running, large-scale applications in production.

Catalyst Optimizer In Spark



- The Catalyst optimizer in Apache Spark is a critical component that enhances query performance by optimizing and transforming high-level SQL queries or DataFrame operations into an efficient execution plan.
- Here's a concise overview of how it works:
 1. SQL Query / DataFrame:
 - The process begins with an input SQL query or DataFrame operation.
 2. Unresolved Logical Plan:
 - The query is parsed and converted into an unresolved logical plan.
 - This plan represents the query's structure but lacks details about table names, columns, and functions.
 3. Catalog:
 - The Catalog resolves table and column names, creating a logical plan.
 - It also validates schema consistency.
 4. Logical Optimization:
 - The logical optimizer applies various rules to simplify and optimize the logical plan.

- It rewrites expressions, eliminates unnecessary operations, and pushes down filters.
 - 5. Optimized Logical Plan:
 - The result is an optimized logical plan that represents the query more efficiently.
 - 6. Physical Planning:
 - The physical planner generates multiple physical plans based on the optimized logical plan.
 - These plans differ in execution strategies (e.g., join algorithms, partitioning).
 - 7. Selected Physical Plan:
 - The query planner selects the best physical plan based on cost estimation.
 - Factors include data distribution, available resources, and parallelism.
 - 8. Code Generation:
 - The chosen physical plan undergoes code generation to produce Resilient Distributed Datasets (RDDs).
 - RDDs are the building blocks for distributed computation.
-

Adaptive Query Execution (AQE)

- Adaptive Query Execution (AQE) is a feature in Apache Spark that dynamically optimizes query execution plans at runtime based on actual data statistics and runtime metrics. AQE improves performance and reliability by adjusting the execution plan as the query progresses.
- 1. Dynamic Coalescing of Shuffle Partitions
 - Purpose: Reduces the overhead of small shuffle partitions by combining them into larger ones, improving task execution efficiency.
 - How It Works:
 - Initial Plan: Shuffle operation may create many small partitions.
 - Runtime Statistics: Spark collects runtime statistics on partition sizes.
 - Adaptive Adjustment: Combines small partitions into larger ones based on statistics, reducing the number of tasks.
 - Example:
 - Initially creates 200 partitions.
 - AQE coalesces small partitions into 50 larger partitions.
 - Results in fewer tasks with larger data chunks, improving performance.
 - Configuration:
 - Property: `spark.sql.adaptive.coalescePartitions.enabled`
 - Default: `true`

2. Dynamic Optimizing of Join Strategies

- Purpose: Chooses the most efficient join strategy based on the actual size of the data being joined, optimizing join operations.
- How It Works:
 - Initial Plan: Plans a specific join strategy (e.g., shuffle join).
 - Runtime Statistics: Collects statistics about the datasets' sizes.
 - Adaptive Adjustment: Switches to a more efficient join strategy (e.g., broadcast join) if one dataset is small.
- Example:
 - Initially plans a shuffle join.
 - AQE detects one table is small and switches to a broadcast join.
 - Reduces shuffle overhead and improves join performance.
- Configuration:
 - Property: `spark.sql.adaptive.join.enabled`
 - Default: `true`

3. Handling Skewed Joins

- Purpose: Addresses data skew in join operations by dynamically adjusting the execution plan to balance the load more evenly across the cluster.
- How It Works:
 - Initial Plan: Plans a standard join operation.
 - Runtime Statistics: Detects skewed data distributions.
 - Adaptive Adjustment: Adjusts the execution plan, such as by splitting large partitions or changing join strategy to balance the load.
- Example:
 - Initially plans a standard join.
 - AQE detects skewed data and adjusts the plan by splitting large partitions.
 - Balances the load and avoids stragglers, improving performance.
- Configuration:
 - Property: `spark.sql.adaptive.skewJoin.enabled`
 - Default: `true`

Feature	Purpose	How It Works	Configuration Property	Default
Dynamic Coalescing of Shuffle Partitions	Reduces overhead of small shuffle partitions.	Combines small partitions into larger ones based on runtime stats.	<code>spark.sql.adaptive.coalescePartitions.enabled</code>	TRUE
Dynamic Optimizing of Join Strategies	Chooses the most efficient join strategy based on data size.	Switches join strategy (e.g., shuffle join to broadcast join).	<code>spark.sql.adaptive.join.enabled</code>	TRUE
Handling Skewed Joins	Addresses data skew by balancing	Adjusts execution plan to handle	<code>spark.sql.adaptive.skewJoin.enabled</code>	TRUE

	load across the cluster.	skewed data distributions.		
--	--------------------------	----------------------------	--	--

Cache and Persist in Apache Spark

- Caching and persisting are mechanisms in Apache Spark used to store intermediate results of computations in memory or on disk, which helps speed up iterative algorithms and interactive data exploration. Both methods help avoid recomputation of data and improve performance, but they have some differences.

1. What is Cache?

- Cache is a method in Spark that stores the DataFrame or RDD using the default storage level, which is MEMORY_ONLY.
- Usage:
 - Frequently used when you need to store the data in memory to speed up subsequent actions on that data.
 - Ideal for datasets that fit comfortably in memory.
- Default Storage Level: MEMORY_ONLY
 - Stores the data in memory as deserialized Java objects.
 - If there is insufficient memory, some partitions will not be cached and will be recomputed when needed.

2. What is Persist?

- Persist is a method in Spark that allows you to specify the storage level explicitly, offering more flexibility compared to cache.
- Usage:
 - Provides more control over how and where the data is stored.
 - Suitable for large datasets that may not fit entirely in memory.
- Common Storage Levels:
 - MEMORY_ONLY: Stores RDD as deserialized Java objects in the JVM. If it doesn't fit in memory, recomputation is needed.
 - MEMORY_AND_DISK: Stores RDD as deserialized Java objects in the JVM. If it doesn't fit in memory, partitions are spilled to disk.
 - MEMORY_ONLY_SER: Stores RDD as serialized Java objects (one byte array per partition). This is more space-efficient.
 - MEMORY_AND_DISK_SER: Similar to MEMORY_ONLY_SER but spills to disk if it doesn't fit in memory.
 - DISK_ONLY: Stores RDD partitions only on disk.
 - OFF_HEAP: (Experimental) Stores the RDD partitions off-heap.

Feature	Cache	Persist
---------	-------	---------

Default Storage Level	MEMORY_ONLY	User-defined (e.g., MEMORY_AND_DISK)
Usage	Simple and quick caching with default level	Flexible, allowing various storage levels
Syntax	<code>rdd.cache()</code> , <code>df.cache()</code>	<code>rdd.persist(StorageLevel.XYZ)</code> , <code>df.persist(StorageLevel.XYZ)</code>
Customization	No	Yes
Performance	Depends on MEMORY_ONLY storage level	Depends on chosen storage level
Resource Utilization	May lead to recomputation if memory is insufficient	Can spill to disk or use serialization to save space

Salting in Apache Spark

- Salting is a technique used in Apache Spark to handle data skew, which occurs when a disproportionate amount of data is assigned to a few partitions. This imbalance can lead to inefficient resource utilization and extended job completion times.
- Data Skew?
 - Data skew happens when the distribution of data across partitions is uneven. In such cases, some partitions hold more data than others, leading to performance bottlenecks. This is common in operations like joins, group by, and aggregations where certain keys have a much higher frequency than others.
- What is Salting?
 - Salting involves adding a "salt" value to the keys in a dataset before performing operations like joins or aggregations. This "salt" value helps distribute the data more evenly across partitions.
- Salting Technique
 - Identify Skewed Keys: Determine which keys are causing the skew by analyzing the data distribution.
 - Add Salt: Modify the skewed keys by appending a random or sequential salt value, creating multiple variations of the key.
 - Perform Operation: Carry out the join or aggregation operation on the salted dataset.
 - Remove Salt: After the operation, remove the salt to restore the original keys.
- Steps in Salting
 - Identify Skewed Keys:
 - Analyze the distribution of keys to identify the skewed keys.
 - Count the occurrences of each key to find those that are disproportionately frequent.
 - Add Salt:

- Add a salt value to the keys in both datasets.
- The salt can be random or sequential, creating multiple variations of the skewed key.
- For example, a skewed key "A" can become "A_1", "A_2", "A_3", etc.
- Perform Operation:
 - Perform the join or aggregation operation using the salted keys.
 - This ensures data is distributed more evenly across partitions.
- Remove Salt:
 - After the operation is completed, remove the salt from the keys.
 - This step restores the keys to their original form, maintaining data integrity.
- Benefits of Salting
 - Balanced Workload: Distributes the workload more evenly across partitions, preventing bottlenecks.
 - Improved Performance: Reduces the time taken for operations by avoiding skew-induced delays.
 - Efficient Resource Utilization: Ensures effective use of CPU and memory resources, leading to faster processing.
- Considerations
 - Increased Data Size: Adding salt increases the size of the data, which might lead to higher memory and storage requirements.
 - Complexity: Introduces additional complexity in data processing pipelines.
 - Key Cardinality: Salting is effective only if the number of unique keys after salting is significantly larger. Too many salted keys could lead to other performance issues.

Spark Optimization Techniques

Technique	Description	Cause	Implementation
Data Serialization	Use Kryo serialization instead of Java serialization	Java serialization is slower and less compact	<code>conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")</code>
Proper Partitioning	Ensure even distribution of data across partitions	Uneven partitions lead to data skew and performance bottlenecks	<code>rdd.repartition(10)</code> (increase partitions), <code>rdd.coalesce(2)</code> (decrease partitions)
Caching and Persistence	Store intermediate results in memory or disk	Recomputing intermediate results is inefficient	<code>rdd.cache()</code> , <code>df.persist(StorageLevel.MEMORY_AND_DISK)</code>
Broadcast Variables	Broadcast small lookup tables to all executors	Repeatedly sending data to executors is inefficient	<code>broadcastVar = sc.broadcast(smallData)</code>

Reduce Shuffling	Minimize data shuffling	Shuffling is expensive in terms of time and resources	Use reduceByKey instead of groupByKey, avoid wide transformations
Join Optimizations	Use broadcast joins for small datasets	Regular joins can cause significant shuffling	largeDF.join(broadcast(smallDF), "key")
Executor Memory Management	Allocate sufficient memory to executors	Insufficient memory leads to out-of-memory errors	--executor-memory 4G
SQL Query Optimization	Use filter operations and partition pruning	Reading unnecessary data increases processing time	df.filter("age > 30"), ensure partition pruning is enabled
Avoid Wide Transformations	Use narrow transformations like map and filter	Wide transformations require shuffling	rdd.map(lambda x: x * 2)
Use Efficient Data Formats	Use columnar storage formats like Parquet and ORC	Inefficient storage formats lead to slower read/write operations	df.write.parquet("path/to/parquet")
Optimize UDFs	Use Pandas UDFs or built-in functions instead of regular UDFs	Regular UDFs can be slow and inefficient	@pandas_udf("int") def add_one(x: pd.Series) -> pd.Series: return x + 1
Configuration Tuning	Enable dynamic allocation to manage executor allocation based on workload	Static allocation can lead to resource underutilization	--conf spark.dynamicAllocation.enabled=true
Adaptive Query Execution (AQE)	Dynamically optimizes query plans at runtime	Static query plans may not be optimal	conf.set("spark.sql.adaptive.enabled", "true")
Resource Management	Use resource managers like YARN, Mesos, or Kubernetes	Inefficient resource allocation	Configure Spark to use resource managers for efficient resource allocation
Compression	Compress serialized RDDs to save memory	Uncompressed data consumes more memory	conf.set("spark.rdd.compress", "true")

Spark UI

- The Spark UI is a web-based interface provided by Apache Spark to monitor and debug Spark applications. It offers detailed insights into the performance, resource utilization, and execution details of Spark jobs running on a cluster.
- Key Features of Spark UI
 1. Job Progress Tracking:
 - Displays the progress of each Spark job, including stages and tasks.

- Provides information on job completion status, stages completed, and tasks executed.
 - 2. Stages and Tasks View:
 - Breaks down jobs into stages, showing tasks within each stage.
 - Allows tracking of task execution times, failures, and resource usage (CPU, memory).
 - 3. Environment Details:
 - Shows Spark and Hadoop configuration details.
 - Lists environment variables and properties used by Spark.
 - 4. Storage Tab:
 - Provides insights into cached RDDs (Resilient Distributed Datasets).
 - Displays storage levels, sizes, and whether they are in memory or on disk.
 - 5. SQL and RDD Operations:
 - Lists operations performed on RDDs and DataFrames.
 - Shows execution plans for SQL queries, including optimizations.
 - 6. Event Timeline:
 - Visualizes the timeline of events during job execution.
 - Includes information on scheduling delays, data shuffling, and task execution phases.
 - 7. Executors and Tasks View:
 - Details about each executor in the cluster.
 - Shows executor logs, memory usage, and CPU metrics.
 - 8. DAG Visualization:
 - Graphical representation of Directed Acyclic Graph (DAG) for jobs.
 - Illustrates dependencies between stages and tasks.
 - Accessing Spark UI
 - Standalone Mode: Typically accessible on port 4040 (<http://localhost:4040>) of the machine where the driver is running.
 - YARN Mode: Available through the YARN Resource Manager UI (<http://yarn-cluster-url:8088/cluster>).
 - Mesos Mode: Accessed via the Mesos Master UI (<http://mesos-master-url:5050>).
 - Benefits of Spark UI
 - Performance Tuning: Helps identify bottlenecks, optimize resource usage, and tune Spark configurations.
 - Debugging: Facilitates troubleshooting by providing detailed logs, metrics, and visualization of job execution.
 - Monitoring: Real-time monitoring of Spark applications to ensure efficient data processing.
-

Shared Variables in Apache Spark

- Shared variables are crucial for distributing data and aggregating results efficiently across a Spark cluster.
- There are primarily two types of shared variables in Spark: broadcast variables and accumulators.
 1. Broadcast Variables
 - Purpose: Efficiently distribute read-only data (e.g., lookup tables) across all nodes in a Spark cluster.
 - Benefits:
 - Reduces network overhead by distributing data once per node.
 - Improves performance for operations using common, large datasets.
 2. Accumulators
 - Purpose: Aggregate values from worker nodes back to the driver program, for metrics or counters.
 - Benefits:
 - Efficiently counts or sums values across distributed tasks.
 - Provides a mechanism for accumulating metrics or custom statistics.
- Considerations
 1. Broadcast Variables: Ensure data is immutable and suitable for distribution across all nodes.
 2. Accumulators: Use for associative and commutative operations to maintain accuracy across distributed tasks.

Feature	Description	Use Case	Implementation
Broadcast Variables	Efficiently distribute read-only data across all nodes in a Spark cluster.	Distributing lookup tables or configuration data.	<code>broadcastVar = sc.broadcast(data)</code>
Accumulators	Aggregate values from worker nodes back to the driver program.	Counting occurrences or summing values across tasks.	Initialize with <code>sc.accumulator(initial_value)</code> , used in foreach operations.

Storage Level in Spark

- In Apache Spark, the storage level determines how RDDs (Resilient Distributed Datasets) or DataFrames are stored in memory and/or on disk during the execution of a Spark job. These storage levels control the persistence of data across various nodes in a cluster, optimizing performance for repeated access to the data. Here's an overview of the different storage levels in Spark:

1. MEMORY_ONLY

- Description: Stores the RDD/DataFrame in memory as deserialized Java objects. If the memory is insufficient, Spark will evict older data to make room for new data.
- Use Case: Ideal when you need to access the data frequently and can afford to lose some data if the memory overflows.
- Persistence: Memory (RAM) only.
- Fault Tolerance: No fault tolerance; data is recomputed if lost.
- Serialization: Data is stored as deserialized objects.
- Command: `StorageLevel.MEMORY_ONLY`

2. MEMORY_ONLY_SER

- Description: Similar to MEMORY_ONLY, but stores the RDD/DataFrame in memory as serialized objects. This reduces the memory footprint but increases CPU overhead due to serialization and deserialization.
- Use Case: Useful when you need to store more data in memory and are willing to trade off CPU performance.
- Persistence: Memory (RAM) only.
- Fault Tolerance: No fault tolerance; data is recomputed if lost.
- Serialization: Data is stored as serialized objects.
- Command: `StorageLevel.MEMORY_ONLY_SER`

3. MEMORY_AND_DISK

- Description: Stores the RDD/DataFrame in memory as deserialized objects. If the data does not fit in memory, it spills the overflow to disk.
- Use Case: Suitable for large datasets that do not fit entirely in memory but need to be accessed frequently.
- Persistence: Memory (RAM) and Disk.
- Fault Tolerance: Fault-tolerant; if memory is insufficient, data is saved to disk.
- Serialization: Data is stored as deserialized objects in memory.
- Command: `StorageLevel.MEMORY_AND_DISK`

4. MEMORY_AND_DISK_SER

- Description: Similar to MEMORY_AND_DISK, but stores the RDD/DataFrame as serialized objects both in memory and on disk. This reduces the memory footprint.
- Use Case: Useful when you have very large datasets and need to optimize memory usage while still maintaining persistence.
- Persistence: Memory (RAM) and Disk.

- Fault Tolerance: Fault-tolerant; data spills to disk when memory is insufficient.
- Serialization: Data is stored as serialized objects both in memory and on disk.
- Command: **StorageLevel.MEMORY_AND_DISK_SER**

5. DISK_ONLY

- Description: Stores the RDD/DataFrame only on disk. No data is kept in memory.
- Use Case: Ideal when memory is highly constrained or the dataset is too large to fit in memory.
- Persistence: Disk only.
- Fault Tolerance: Fault-tolerant; data is persisted on disk.
- Serialization: Data is stored as serialized objects on disk.
- Command: **StorageLevel.DISK_ONLY**

6. MEMORY_ONLY_2, MEMORY_AND_DISK_2, etc.

- Description: These storage levels replicate the RDD/DataFrame across two nodes in the cluster. This provides higher fault tolerance since the data is stored redundantly on multiple nodes.
- Use Case: Use when fault tolerance is critical and you can afford to double the storage cost.
- Persistence: Memory and/or Disk with replication.
- Fault Tolerance: High, with redundancy across multiple nodes.
- Serialization: Similar to their non-replicated counterparts.
- Command: **StorageLevel.MEMORY_ONLY_2**,
StorageLevel.MEMORY_AND_DISK_2, etc.

Storage Level	Persistence	Serialization	Fault Tolerance	Use Case
MEMORY_ONLY	Memory (RAM)	Deserialized	No	Frequent access to data, memory available
MEMORY_ONLY_SER	Memory (RAM)	Serialized	No	Optimize memory usage, willing to trade CPU performance
MEMORY_AND_DISK	Memory + Disk	Deserialized in memory	Yes	Large datasets, frequent access, partial memory fit
MEMORY_AND_DISK_SER	Memory + Disk	Serialized	Yes	Very large datasets, optimize memory usage
DISK_ONLY	Disk	Serialized	Yes	Memory-constrained environments
MEMORY_ONLY_2	Memory (2 nodes)	Deserialized	Yes (with replication)	High fault tolerance, memory available
MEMORY_AND_DISK_2	Memory + Disk (2 nodes)	Deserialized	Yes (with replication)	High fault tolerance, large datasets

groupByKey() vs reduceByKey() vs aggregateByKey() vs sortBy() vs sortByKey()

Operation	Purpose	Shuffling	Performance	Use Case
groupByKey()	Groups values for each key into a collection (e.g., list)	Full shuffle required	Slow and memory-intensive	When you need to perform operations on all values for a key
reduceByKey()	Reduces values for each key using a commutative, associative function	Partial shuffle (local reduce first)	Faster than groupByKey()	When you need to reduce data for each key (e.g., summing)
aggregateByKey()	Aggregates values with custom logic using two functions	Partial shuffle (local aggregation first)	Flexible, more complex logic	When you need complex aggregation logic
sortBy()	Sorts an RDD based on a specified key or transformation	Full shuffle required	Expensive (due to shuffle)	When you need to sort based on a specific attribute or function
sortByKey()	Sorts a pair RDD by key	Full shuffle required	Expensive (due to shuffle)	When you need to sort a pair RDD by key

map vs flatMap

Aspect	map	flatMap
Operation	Applies a function to each element of the RDD and returns a new RDD.	Applies a function to each element of the RDD, but returns a flattened result.
Output	Produces an RDD where each input element is transformed into a single output element.	Produces an RDD where each input element can be transformed into zero or more output elements.
Result Type	One-to-one transformation (one element in, one element out).	One-to-many transformation (one element in, multiple elements out).
Usage	Used when you want to map each input element to a single output element.	Used when you want to map each input element to multiple output elements or a flat structure.
Example	<code>rdd.map(x => x * 2)</code> convert each element to its double.	<code>rdd.flatMap(x => x.split(" "))</code> splits each line into words, resulting in a flattened list of words.
Performance	Generally has less overhead compared to	Can be more computationally expensive due

	flatMap, as it doesn't involve flattening.	to the additional step of flattening the result.
--	--	--

Read & Write Mode

Read Mode

- The read mode refers to how data is read from different sources into DataFrames. This includes specifying the format, options, and modes for reading the data.

Format	Syntax	Key Options	Use Case
CSV	<code>.format("csv")</code>	header, inferSchema, sep	Reading CSV files with headers and type inference
JSON	<code>.format("json")</code>	multiLine	Reading JSON files with possible multiline records
Parquet	<code>.format("parquet")</code>	-	Efficient columnar storage format
ORC	<code>.format("orc")</code>	-	Efficient columnar storage format
Avro	<code>.format("avro")</code>	-	Data serialization format
Delta	<code>.format("delta")</code>	-	ACID transactions and time travel
JDBC	<code>.format("jdbc")</code>	url, dbtable, user, password	Reading from relational databases

Write Mode

- The write mode specifies how data is written to an external storage system or file system.
- Different write modes determine the behavior when data already exists at the target location.

Mode	Description	Effect on Existing Data	Use Case
overwrite	Replaces existing data	Deletes existing data and writes new data	Use when you want to replace all existing data
append	Adds new data to existing data	Appends new data to the existing data	Use when continuously adding data

ignore	Ignores write operation if data exists	No action if data exists	Use to avoid overwriting existing data
error	Throws an error if data already exists	Raises an exception	Use to enforce strict data handling rules